

PhiBase Arithmetic: A Rational Number System for Exact Computation and Structural Design

April 8, 2025

Introduction

When working with three dimensional lattice points generated by the Golden Ratio, the use of irrational quantities gets in the way of the underlying symmetries. We now introduce a system that separates out the irrational and results in a simple integer based arithmetic.

This document presents **PhiBase arithmetic**, an exact computational framework built upon the number system $\mathbb{Q}(\phi)$, where $\phi = \frac{1+\sqrt{5}}{2}$ (the golden ratio). PhiBase arithmetic entirely avoids floating-point operations by representing every number as an exact combination of integers and ϕ .

Each coordinate (X, Y, Z) in three-dimensional space is represented explicitly as a PhiBase pair of integers:

$$X = (p_x, x_x), \quad Y = (p_y, x_y), \quad Z = (p_z, x_z),$$

corresponding exactly to:

$$X = p_x \cdot \phi + x_x, \quad Y = p_y \cdot \phi + x_y, \quad Z = p_z \cdot \phi + x_z.$$

PhiBase Arithmetic from Six-Basis Coordinates

In the established six-basis structural system, each 3D coordinate (X, Y, Z) is derived from a six-vector of integer coefficients $[a, b, c, d, e, f]$. The explicit transformation to Cartesian coordinates is originally given by:

$$\begin{aligned} X &= \phi(a + b) + (e - f) \\ Y &= (c - d) + \phi(e + f) \\ Z &= (a - b) + \phi(c + d). \end{aligned}$$

These coordinates inherently fit within the PhiBase system, as they are rational-linear combinations of integers and ϕ . This direct relationship leads naturally to the PhiBase representation.

Explicit PhiBase Conversion

Converting from a six-vector directly to PhiBase form yields integer pairs as follows:

$$\begin{aligned} X &= (a + b) \cdot \phi + (e - f) &\rightarrow (p_x, x_x) &= (a + b, e - f) \\ Y &= (e + f) \cdot \phi + (c - d) &\rightarrow (p_y, x_y) &= (e + f, c - d) \\ Z &= (c + d) \cdot \phi + (a - b) &\rightarrow (p_z, x_z) &= (c + d, a - b). \end{aligned}$$

Note that each of $p_x, x_x, p_y, x_y, p_z, x_z$ is explicitly an integer, entirely avoiding floating-point computation.

Example of PhiBase Conversion

Consider the six-vector example:

$$[a, b, c, d, e, f] = [2, -1, 3, 0, -1, 1].$$

The PhiBase Cartesian coordinates become:

$$\begin{aligned} p_x &= a + b = 2 + (-1) = 1, & x_x &= e - f = -1 - 1 = -2 \\ p_y &= e + f = -1 + 1 = 0, & x_y &= c - d = 3 - 0 = 3 \\ p_z &= c + d = 3 + 0 = 3, & x_z &= a - b = 2 - (-1) = 3. \end{aligned}$$

Thus, the exact PhiBase representation is:

$$X = (1, -2), \quad Y = (0, 3), \quad Z = (3, 3),$$

which explicitly translates to:

$$X = 1 \cdot \phi - 2, \quad Y = 0 \cdot \phi + 3, \quad Z = 3 \cdot \phi + 3.$$

Benefits of PhiBase Arithmetic

The advantages of adopting the PhiBase representation are substantial:

- **Perfect Accuracy:** Eliminates rounding errors completely.
- **Simple Integer Arithmetic:** Direct and exact addition/subtraction of integer pairs.
- **Memoizable Operations:** Multiplication and division can be efficiently cached for repetitive computations.
- **Clear Debugging and Readability:** Explicit integer pairs facilitate intuitive understanding and error tracing.

Implementation and Next Steps

To effectively leverage PhiBase arithmetic:

1. Explicitly adopt PhiBase representation for internal computational code.
2. Store and compute each 3D coordinate explicitly as PhiBase integer pairs.
3. Replace floating-point arithmetic entirely with PhiBase pair operations.
4. Defer floating-point conversion strictly for visualization or final external interfaces.

This strategy maintains exactness and robustness, ensuring that computational precision and structural integrity are consistently preserved.